# Maxkedd - Project Portfolio Page

## Overview

EduReserve is a Java command-line application for managing exam and lecture venue bookings. My functional contributions include booking creation and validation, undo/redo, the initial application structure, and booking correctness checks.

## Summary of contributions

### New feature: Booking creation (`book`)

- **What it does:** Creates a named exam or lecture booking and displays its details.
- **Justification:** Lets users record a booking's purpose, name, venue and timing in one command.
- **Highlights:** Built the initial `Task`, `Booking` and `BookingList` classes, including add, retrieve, remove and count operations. Added `n/BOOKING_NAME`, `getName()` and name display. Extracted execution into `BookCommand`, reusing the parser and delegating from `Ui`.
- **Code:** BookCommand, Booking, BookingList. Commits: b3ef19f, e87b63c, 6c28965.

### New feature: Undo (`/undo`)

- **What it does:** Reverses successful `book` and `cancel` commands, one at a time, restoring the original booking order.
- **Justification:** Allows recovery from accidental additions or cancellations.
- **Highlights:** Implemented session history with booking snapshots and original command text. Integrated parsing and dispatch, repeated undo, empty-history handling and argument validation. Failed and read-only commands create no history entries. Made the existing cancellation operation undoable and restored the retained history instance after a merge removed it.
- **Code:** UndoCommand, Ui. Commits: ba3ce61, 14e325c.

### New feature: Redo (`/redo`)

- **What it does:** Reapplies the most recently undone command and displays that command's text.
- **Justification:** Lets users reverse an accidental undo without entering the original command again.
- **Highlights:** Added before/after snapshots and a redo stack shared through `UndoCommand`. Redone entries return to undo history. Successful new `book` or `cancel` commands clear redo history; failed and read-only commands preserve it. Integrated parsing, dispatch, repeated redo, empty-history handling and argument validation.
- **Code:** RedoCommand, UndoCommand. Commit: 5315d6f.

### Other functional contributions

- **Parsing and errors:** Implemented case-insensitive command recognition, unknown-command rejection, required prefix order and nonblank fields. Restricted reasons to exam/lecture while preserving spaces within values. Added `InvalidCommandException` and error/help output that lets users continue. Parser; b3ef19f, e87b63c.
- **Booking correctness:** Added case-insensitive duplicate-name rejection. Moved the existing clash check before insertion and removed duplicate parsing, insertion and confirmation from `Ui`. The clash algorithm and original cancellation feature were teammates' work; my contributions were integration, correction and undo support. BookCommand; ebf1c6f, ba3ce61.
- **Application foundation:** Created the initial entry point, shared booking state, console loop and command help. Added blank-line skipping, `bye` exit and end-of-input handling. EduReserve, HelpText; b3ef19f.
- **Invariant checks:** Added booking-field and parser-input assertions, plus non-null, size and append checks in `BookingList.add`. Enabled assertions for Gradle `run`; user-facing validation still uses exceptions. BookingList, build.gradle; a16e04e.